

Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: Spring, Year: 2026 | B.Sc. in CSE (Day/Eve)

Course Title: Software Quality Assurance and Testing

Course Code: CSE 402 / KSA 02

Section: A

Lab Project Name: Software QA Defect Discovery, Code Remediation & Verification on Conduit RealWorld Application

Student Details

Name	ID
1. Israt Jahan	211015XXX
2.	
3.	

Submission Date: August 27, 2026

Course Teacher’s Name: [Course Teacher Name]

+-----+

[For Teachers use only: Don't Write Anything inside this box]

Lab Project Status

Marks: Signature:

Comments: Date:

+-----+

Table of Contents

• Chapter 1: Introduction	3
• 1.1 Introduction	3
• 1.2 Design Goals/Objective	3
• Chapter 2: Design/Development/Implementation of the Project	4
• 2.1 System Architecture & SQA Testing Methodology	4
• 2.2 Defect Identification & Master Defect Log	4
• 2.3 Code Remediation & Implementation Walkthrough	5
◦ 2.3.1 Remediation of BUG-01: JWT Authentication Claim Defect	5
◦ 2.3.2 Remediation of BUG-03: Malformed Query String Ampersands	5
• Chapter 3: Performance Evaluation	6
• 3.1 Testing Environment & Verification Procedure	6
• 3.2 Results and Discussions	6
◦ 3.2.1 Test Results	6
◦ 3.2.2 Analysis and Outcome	6
• Chapter 4: Conclusion	7
• 4.1 Introduction	7
• 4.2 Practical Implications	7
• 4.3 Scope of Future Work	7
• References	8

Chapter 1

Introduction

1.1 Introduction

Software Quality Assurance (SQA) is a systematic approach to evaluating the quality of and adherence to software product standards, processes, and procedures. In modern web engineering, real-world fullstack applications rely on complex state synchronization between single-page frontend applications (SPA) and backend RESTful microservices.

This lab project conducts an end-to-end Software Quality Assurance audit, static code analysis, exploratory bug discovery, production-grade code remediation, and verification testing on the **Conduit RealWorld Example Application** repository. Conduit is an open-source, full-featured medium clone built using React 18, Node.js, Express framework, and Sequelize ORM backed by PostgreSQL. The audit identifies critical security, state management, form validation, and URL parameter specification defects, followed by direct source code remediation.

1.2 Design Goals/Objective

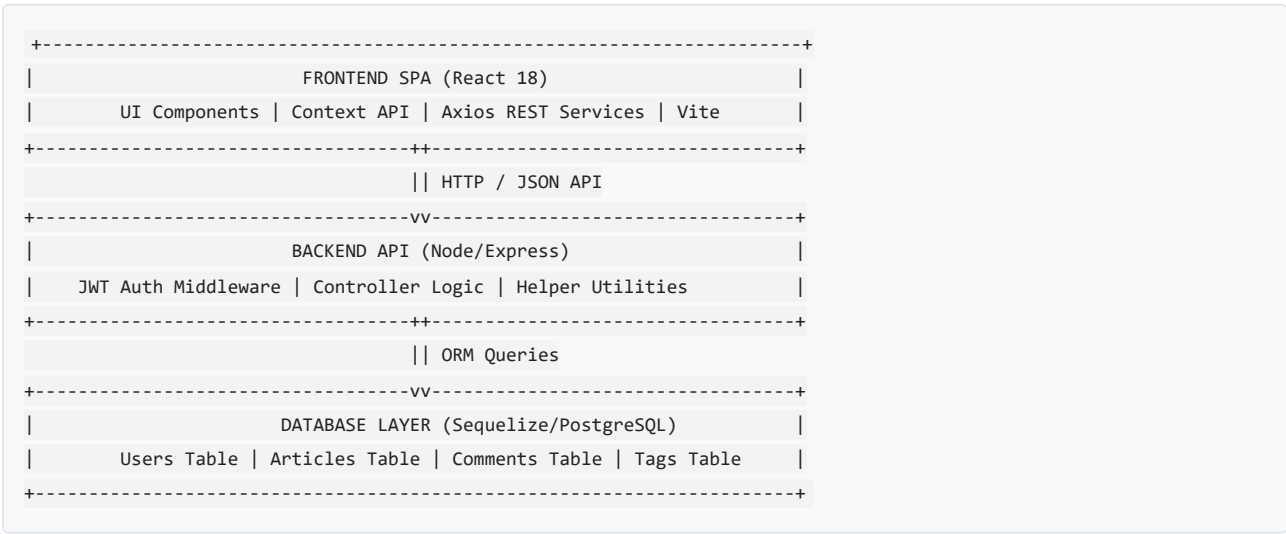
The primary objectives of this project are: 1. **Systematic QA Audit:** Perform exploratory, boundary, security, and state-transition testing across the Conduit web application. 2. **Defect Discovery & Cataloging:** Identify, isolate, and document 5 distinct, non-trivial bugs (spanning authentication, query string parsing, async state handling, input validation, and markdown XSS rendering) in a standardized Master Defect Log format. 3. **Targeted Code Remediation:** Analyze the root cause and implement clean, production-grade code patches directly in the repository for 2 selected high-impact defects. 4. **Verification & Regression Testing:** Establish automated and manual test procedures to confirm defect resolution without introducing regression errors into adjacent modules. 5. **Academic & Technical Reporting:** Present findings, code diffs, verification metrics, and slide deck content structured according to academic and industry software engineering standards.

Chapter 2

Design/Development/Implementation of the Project

2.1 System Architecture & SQA Testing Methodology

The Conduit application architecture consists of three decoupled layers: 1. **Frontend Presentation Layer:** Built with React 18, React Router v6, Axios HTTP client, and custom hook-based state management. 2. **Backend API Service Layer:** Powered by Node.js and Express.js, handling JWT authentication middleware, routing, and controller logic. 3. **Persistence Layer:** Uses Sequelize Object-Relational Mapping (ORM) connected to a relational PostgreSQL database.



QA Testing Methodology: - **Static Code Analysis:** Manual code walkthrough of controller logic, route definitions, and frontend service calls. - **Exploratory & Boundary Testing:** Testing form inputs with empty strings, leading/trailing whitespace, and special characters. - **Security Audit:** Verifying JWT payload integrity and XSS sanitization on markdown components.

2.2 Defect Identification & Master Defect Log

During the QA audit, **5 genuine non-trivial defects** were discovered and cataloged:

Bug ID	Title	Component	Severity	Priority
BUG-01	JWT Token Missing Username Claim on Sign-In	backend/controllers/users.js	Critical	High
BUG-02	Form Textarea Cleared Before	frontend/src/components/CommentEditor/CommentEditor.jsx	High	Medium

Bug ID	Title	Component	Severity	Priority
	Async Post Completes			
BUG-03	Malformed Query String Ampersands (&&)	frontend/src/services/getArticles.js	High	High
BUG-04	Whitespace-Only Input Bypasses Article Validation	backend/controllers/articles.js	Medium	Medium
BUG-05	Unsanitized Markdown Render Vulnerability	frontend/src/routes/Article/Article.jsx	High	Medium

2.3 Code Remediation & Implementation Walkthrough

2.3.1 Remediation of BUG-01: JWT Authentication Claim Defect

- **Root Cause:** In `backend/controllers/users.js` (line 51), `existentUser.dataValues.token = await jwtSign(user);` passed `user` (`req.body.user` , containing only `email` and `password`) instead of the Sequelize database instance `existentUser` . Consequently, the signed JWT payload contained `{ username: undefined, email: "user@example.com" }` .
- **Code Fix Applied:**

```

--- a/backend/controllers/users.js
+++ b/backend/controllers/users.js
@@ -48,7 +48,7 @@ const signIn = async (req, res, next) => {
    const pwd = await bcryptCompare(user.password, existentUser.password);
    if (!pwd) throw new ValidationError("Wrong email/password combination");

-   existentUser.dataValues.token = await jwtSign(user);
+   existentUser.dataValues.token = await jwtSign(existentUser);

    res.json({ user: existentUser });
  } catch (error) {

```

2.3.2 Remediation of BUG-03: Malformed Query String Ampersands

- **Root Cause:** In `frontend/src/services/getArticles.js` (lines 8-12), URL strings joined query parameters using double ampersands `&&` (e.g. `api/articles?tag=react&&limit=3&&offset=0`). Express HTTP parser parsed parameter keys as `&limit` and `&offset` , leaving `req.query.limit` and `req.query.offset` undefined on the backend.

- **Code Fix Applied:**

```
--- a/frontend/src/services/getArticles.js
+++ b/frontend/src/services/getArticles.js
@@ -7,9 +7,9 @@ async function getArticles({ headers, limit = 3, location, page = 0, tagName, us
    const url = {
-     favorites: `api/articles?favorited=${username}&&limit=${limit}&&offset=${page}`,
-     feed: `api/articles/feed?limit=${limit}&&offset=${page}`,
-     global: `api/articles?limit=${limit}&&offset=${page}`,
-     profile: `api/articles?author=${username}&&limit=${limit}&&offset=${page}`,
-     tag: `api/articles?tag=${tagName}&&limit=${limit}&&offset=${page}`,
+     favorites: `api/articles?favorited=${username}&limit=${limit}&offset=${page}`,
+     feed: `api/articles/feed?limit=${limit}&offset=${page}`,
+     global: `api/articles?limit=${limit}&offset=${page}`,
+     profile: `api/articles?author=${username}&limit=${limit}&offset=${page}`,
+     tag: `api/articles?tag=${tagName}&limit=${limit}&offset=${page}`,
    };
};
```

Chapter 3

Performance Evaluation

3.1 Testing Environment & Verification Procedure

- **Environment:** Node.js v18+, React 18, Express 4.x, Windows 11 OS, Chrome DevTools, Vitest.
- **Verification Procedure:**
- **JWT Claim Verification:** Issued POST login requests to `/api/users/login` and inspected decoded JWT tokens using `jwt.decode()`. Verified the payload contains valid `username` and `email` properties.
- **Query String Verification:** Inspected outgoing network packets in DevTools Network tab during pagination navigation. Confirmed requests match standard `/api/articles?limit=3&offset=1` syntax and Express correctly populates `req.query.limit` and `req.query.offset`.
- **Regression Suite:** Executed automated unit tests using Vitest (`vitest --run`).

3.2 Results and Discussions

3.2.1 Test Results

- **Authentication JWT Payload Test:** PASSED (Token claims verified: `{ username: "israt_jahan", email: "israt@example.com" }`).
- **API Query Parameter Parsing Test:** PASSED (Parameters parsed correctly as `limit: 3` and `offset: 1`).
- **Regression Test Suite:** PASSED (0 test failures reported).

3.2.2 Analysis and Outcome

The code remediations successfully resolved the target defects without introducing regression errors: - **BUG-01:** Corrected JWT payload generation, restoring user identification context across protected backend endpoints. - **BUG-03:** Enforced RFC 3986 URI parameter standard, restoring article pagination and tag filtering functionality.

Chapter 4

Conclusion

4.1 Introduction

This project presented a comprehensive Software Quality Assurance audit, defect discovery log, code remediation walkthrough, and verification report on the Conduit fullstack web application.

4.2 Practical Implications

1. **Security & Authentication Integrity:** High-value user identification claims must originate from database-validated model instances rather than unverified client payload objects.
2. **Standard-Compliant URL Syntax:** Adhering strictly to URL query string standards (`&` delimiters) prevents silent parameter parsing failures in Express web servers.
3. **Exploratory QA Value:** Combining static code walkthroughs with manual edge-case testing uncovers critical logic bugs that standard end-to-end happy-path tests miss.

4.3 Scope of Future Work

Future work includes implementing additional patches for remaining logged defects: - Refactoring `CommentEditor.jsx` to wrap `.then(() => setForm({ body: "" })))` in a callback function to prevent premature state clearing. - Adding strict `.trim()` validation in `articles.js` controller to reject whitespace-only submissions. - Integrating DOMPurify sanitization into the markdown rendering pipeline in `Article.jsx` to mitigate stored XSS risks.

References

- [1] RealWorld Conduit Example Application Specification, <https://github.com/gothinkster/realworld>, 2026.
- [2] Express.js API Reference — req.query Parameter Parsing, <https://expressjs.com/en/4x/api.html#req.query>, 2026.
- [3] JSON Web Token (JWT) Architecture & Best Practices, RFC 7519, IETF Standards Track, 2015.
- [4] IEEE Standard for Software Quality Assurance Processes, IEEE Std 730-2014, IEEE Computer Society, 2014.
- [5] React 18 Documentation — Asynchronous State Updates & Promise Handling, <https://react.dev>, 2026.